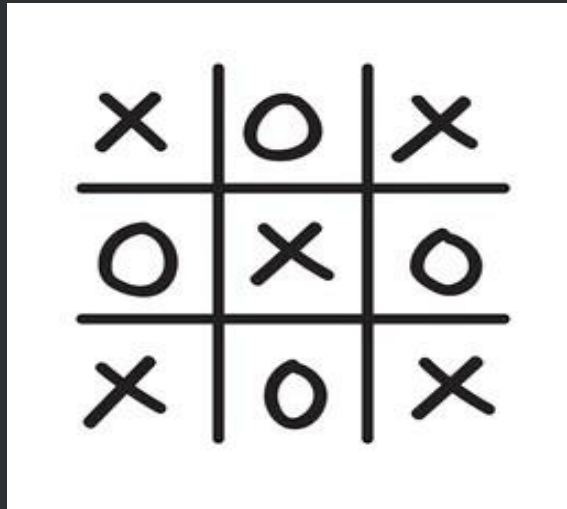


TIC TAC TOE



Java Console Application

Presented by: **Anvesha Mudgal**
Registration Number : 25BAI10607

B.Tech CSE(AIML)
VIT Bhopal University

Introduction & Problem Statement

Introduction

Tic-Tac-Toe is a classic two-player game that demonstrates fundamental programming concepts. This Java implementation showcases how basic programming constructs can create a fully functional, interactive game.

Problem Statement

Design and develop a functional Java program that allows two human players to play Tic-Tac-Toe on a 3×3 grid — verifying moves, updating the board, detecting the winner, and determining draw conditions.



Purpose 1

Learn core Java concepts through practical application



Purpose 2

Understand game logic and control flow



Purpose 3

Implement console-based user interaction

REQUIREMENTS

Project Overview & Requirements

A two-player console-based Tic-Tac-Toe game where players alternate marking positions on a 3×3 grid. The game detects winning patterns and draw conditions automatically.

Functional Requirements

Player Input Module

Accepts moves (1–9) from Player X and Player O

Board Management

Stores and updates the 3×3 grid using arrays; displays board after each turn

Winner / Draw Detection

Checks all 8 winning combinations — 3 rows, 3 columns, 2 diagonals

Non-Functional Requirements



Usability

Clear prompts and simple gameplay



Reliability

Correctly identifies all winning patterns



Performance

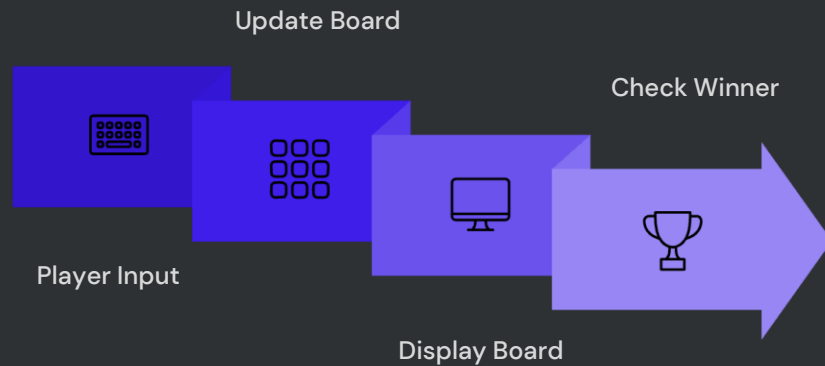
Instant response — lightweight console app



Maintainability

Modular code with separate methods

System Architecture



How It Works

The system follows a **linear game loop structure**. Each iteration of the loop processes one player's turn in sequence:

01

Receive Input

Player enters a position number (1–9) via the console

02

Update & Display

The board array is updated and the current state is printed to console

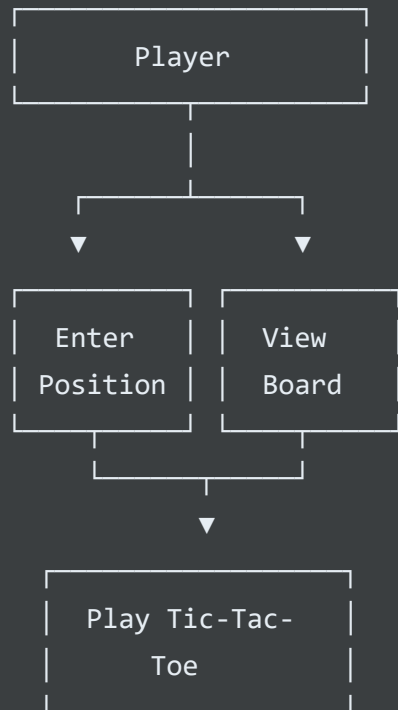
03

Evaluate State

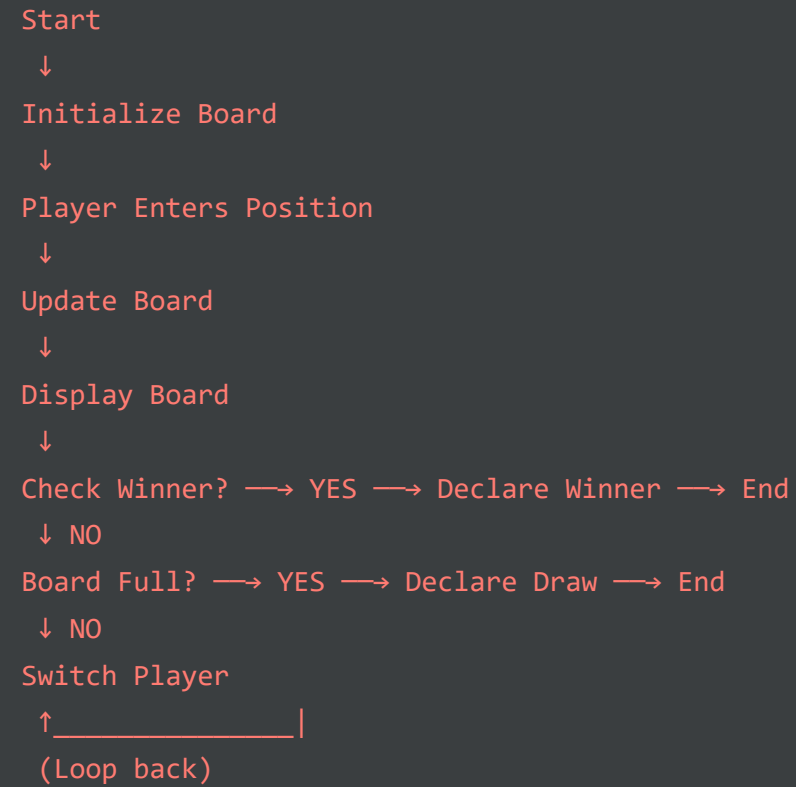
Winner or draw is checked; game continues or terminates accordingly

Design Diagrams

Use Case Diagram



Workflow Diagram



Sequence & Component Design

Sequence Diagram

```
Player → Game    : Enter Position (1-9)
Game  → Board    : Update Cell
Board → Game      : Board Updated
Game  → Display: Show Board
Game  → Logic     : Check Winner
Logic → Game      : Winner / Draw / Continue
Game  → Player    : Display Result
```

Component Design — Project.java

Data Structure

Character array for board state storage

showBoard()

Renders the current board to console

winner()

Checks all 8 winning combinations

add3()

Helper method used by winner checking

main()

Game loop, player switching, and control flow

IMPLEMENTATION

Implementation Details

Core Java Concepts Used

Arrays

1D character array for board positions

Methods

Modular functions for display & win logic

Loops

While/for loops for turns & pattern checks

Conditionals

If-else for move validation & win detection

Scanner

Console input from the user

Winner Checking Logic — 8 Combinations

```
// Rows
if (add3(board[0], board[1], board[2]) == 1) return 1;
if (add3(board[3], board[4], board[5]) == 1) return 1;
if (add3(board[6], board[7], board[8]) == 1) return 1;

// Columns
if (add3(board[0], board[3], board[6]) == 1) return 1;
if (add3(board[1], board[4], board[7]) == 1) return 1;
if (add3(board[2], board[5], board[8]) == 1) return 1;

// Diagonals
if (add3(board[0], board[4], board[8]) == 1) return 1;
if (add3(board[2], board[4], board[6]) == 1) return 1;
```

❏ The game checks **3 rows + 3 columns + 2 diagonals = 8 winning patterns** on every turn.

Results & Testing

Sample Game Output

```
Initial Board:
1 | 2 | 3
-----
4 | 5 | 6
-----
7 | 8 | 9

After Moves:
X | 2 | 3
-----
4 | 0 | 6
-----
7 | 8 | X

Winner: X (diagonal match)
```

Test Cases

Test Case	Condition	Result
Row Win	Three X's in a row	X Wins
Column Win	Three O's in a column	O Wins
Diagonal Win	Three X's diagonally	X Wins
Draw	All 9 positions filled, no winner	Draw
Game Termination	Winner declared or draw reached	Game Ends

Challenges, Learnings & Future Enhancements

Challenges Faced

- Managing alternating player turns correctly
- Checking all 8 winning combinations efficiently
- Handling all game termination edge cases
- Maintaining board state throughout the game

Key Learnings

- Java arrays for data storage and manipulation
- Method design and modular programming
- Loop structures and conditional logic
- Console I/O using the Scanner class

Future Enhancements

1

AI Opponent

Computer player with game strategy

2

Scoreboard

Track wins across multiple game rounds

3

GUI

Java Swing or JavaFX graphical interface

4

Multiplayer

Network-based multiplayer functionality

5

Validation

Input check for already-occupied positions

Winning Patterns & Conclusion

All 8 Winning Patterns

Rows:	Columns:	Diagonals:
X X X	X . .	X . .
. . .	X . .	. X .
. . .	X . .	. . X
. . .	. X .	. . X
X X X	. X .	. X .
. . .	. X .	X . .
. . .	. . X	
. . .	. . X	
X X X	. . X	

Conclusion

This Tic-Tac-Toe project demonstrates how fundamental Java concepts — **arrays, methods, loops, and conditional statements** — can be combined to build a complete, functional console application.

The project provides practical experience in game logic implementation and reinforces core programming principles essential for B.Tech students.

References: Project.java – Main implementation file | Original Project Report – Academic reference

Arrays & Methods Core data & logic structures	Loops & Conditionals Game flow & decision-making	Scanner I/O Console interaction
--	---	------------------------------------